train positioning and movement event data across the network. You can easily subscribe to this public stream of data and try ingesting it into your own application to get an idea of ingesting streaming data.

We can add a sliding window over the stream as we read it to calculate the percentage of train movements that are late in a five minute period as shown in listing 4.5. GitHub includes the full code and instructions on how to run this example.

Listing 4.5 Calculating the percentage of late train movements across the UK rail network in a five minute sliding window.

```
cutoff = datetime.now() - timedelta(minutes=5) #1
while window and window[0][0] < cutoff: #2
  window.popleft()

for m in (json.loads(frame.body) if isinstance(json.loads(frame.body), list)
➥ else [json.loads(frame.body)])[:5]:
  msg_type = m.get('header',{}).get('msg_type','')
  body = m.get('body',{})

  if msg_type == '0003':  #3
    status = body.get('variation_status','')
    train_id = body.get('train_id','')
    print(f"{status} {train_id}")
    window.append((datetime.now(), train_id, status == 'LATE'))

    count += 1
    if count % 25 == 0:  #4
      late_count = sum(1 for _, _, is_late in window if is_late) <>
      on_time_rate = (len(window) - late_count) / len(window) * 100 if
      ➥window else 100
      print(f"5min window: {len(window)} movements, {on_time_rate:.1f}%
      ➥on-time")
```

#1 Set the window to be five minutes in length.
#2 Remove events from the window that are older than five minutes.
#3 Only process train movement events.
#4 Calculate aggregates every 25 events.

4.4.3 API integration

APIs provide structured, on-demand access to data from both internal enterprise systems and external services, making them essential components of digital twin data ingestion strategies. Unlike batch or streaming ingestion that push data on predetermined schedules, APIs enable digital twins to pull specific data when needed, supporting both scheduled collection and event-driven integration patterns. Digital twins typically integrate with:

- *Representational state transfer (REST)* APIs are widespread across many systems due to their simplicity and provision of predictable, cacheable access to well-defined resources.